ClearConsent: BearHacks 2026 Development Document

This document outlines the complete system architecture, technical stack, and distinct role assignments for the development of ClearConsent during the 36-hour BearHacks 2026 hackathon. The division of labor ensures parallel development across the frontend, backend, data processing pipeline, and final presentation.

1. System Architecture & Tech Stack

The application is structured as a responsive mobile web application to allow easy camera access without the friction of app store deployment, supported by a high-performance Python backend.

- **Frontend:** React 19 (Mobile-first responsive design)
- **Backend Server:** FastAPI (Python)
- **OCR Engine:** Google Cloud Vision API
- **Distributed Processing:** Distributive Compute Protocol (DCP)
- **LLM & State Management:** Backboard API (for persistent "Legal Profiles")

2. Division of Labor: Role Assignments

The workload is divided into three distinct engineering pipelines and one dedicated product/demo pipeline to maximize efficiency and minimize merge conflicts.

Developer 1: Frontend & UI/UX (React)

Responsible for the user-facing application, camera integration, and translating complex legal data into intuitive visual indicators.

- **Camera & Upload Pipeline:** Implement HTML5 getUserMedia API for live document scanning and image cropping.
- **Risk Dashboard:** Build the "Traffic Light" UI (Red/Yellow/Green) to display the plain-language risk score.
- **Clause Highlighter:** Create an interface that displays the extracted text and visually flags the "3 most consequential clauses."
- **User Profile:** Build a simple settings page to interface with Backboard, allowing users to

toggle dealbreakers (e.g., "Flag all mandatory arbitration clauses").

Developer 2: Backend API & AI Orchestration (FastAPI)

Responsible for the core server architecture, prompt engineering, and maintaining conversational state.

- **API Routing:** Construct the FastAPI endpoints that bridge the React frontend with the processing pipelines.
- **Backboard Integration:** Connect to the Backboard API to manage the user's persistent "Legal Profile." Ensure the LLM cross-references the current scanned document against the user's stored dealbreakers.
- **Prompt Engineering:** Design the strict system prompts required to force the LLM to output predictable JSON containing the risk score, the 3 key clauses, and the plain-text translation.
- **Error Handling:** Implement graceful degradation if a document is unreadable or an API rate limit is hit.

Developer 3: Document Processing & DCP Pipeline (Python)

Responsible for extracting raw data from the images and handling the heavy computational lifting for multi-page documents.

- **Google Vision Integration:** Implement the Vision API to perform layout-aware OCR. Extract text while preserving structural context (headings, signature blocks).
- **Document Chunking:** Write logic to split long, dense legal documents into logical, overlapping semantic chunks that fit within LLM context windows.
- **DCP Implementation:** Wire the document chunks into the Distributive Compute Protocol. Dispatch the chunks to worker nodes for parallel summarization, drastically reducing the latency of analyzing a 20-page consent form.
- **Data Aggregation:** Stitch the parallelized summaries back together into a cohesive payload to hand off to Developer 2.

Role 4: Demo, Pitch & Storytelling ("The Closer")

Responsible for securing the "Presentation" and "Creativity" criteria points. This role ensures the technical work translates into a winning narrative.

- **The "Break the Norm" Narrative:** Craft the pitch script focusing on the information asymmetry between massive healthcare/legal institutions and everyday people.
- **Video Production:** Record, edit, and narrate the mandatory Demo Video. Focus heavily on a clean, real-time walkthrough of the app functioning.
- **Physical Prop Prep:** Print out highly complex, intimidating medical consent forms to use

as physical props during the live demo and video recording to prove the app works on real paper.

- **Pitch Deck:** Design a 3-5 slide deck covering the problem, the architecture (highlighting Vision, DCP, and Backboard to hit sponsor tracks), and the market viability.

3. Development Timeline (36-Hour Schedule)

Phase	Timeframe	Key Objectives
Phase 1: Skeleton & Setup	Hours 1 - 6	Initialize Git repo. Set up FastAPI and React boilerplate. Verify API keys for Google Vision, Backboard, and DCP. Establish a hardcoded mock JSON response so Frontend can build UI immediately.
Phase 2: Core Processing	Hours 6 - 18	Dev 3 builds Vision OCR and DCP parallelization. Dev 2 finalizes Backboard prompts. Dev 1 completes camera UI. Demo Lead starts drafting pitch narrative.
Phase 3: Integration	Hours 18 - 26	Connect React to FastAPI. Remove mock data and push real Vision API data through the backend pipeline to the frontend. Debug layout issues.
Phase 4: Polish & Pitch	Hours 26 - 36	Code freeze on new features. Fix critical bugs only. Demo Lead records

Phase	Timeframe	Key Objectives
		video. Team practices live pitch. Deploy backend to a simple cloud instance (or run locally via ngrok for demo).

4. Data Handshake Schema (API Contract)

To ensure Dev 1 and Dev 2 can work independently without blocking each other, the team will adhere to this JSON structure for the core analysis endpoint.

```
{
  "document_id": "doc_84729",
  "overall_risk_score": "HIGH", // "LOW", "MEDIUM", "HIGH"
  "summary_plain_english": "This document allows the hospital to charge you for out-of-network providers and waives your right to a jury trial in case of malpractice.",
  "flagged_clauses": [
    {
      "type": "WAIVER_OF_RIGHTS",
      "original_text": "Patient agrees to binding arbitration for any dispute...",
      "translation": "You cannot sue them in court. You must use a private arbitrator.",
      "severity": "CRITICAL"
    },
    {
      "type": "FINANCIAL_LIABILITY",
      "original_text": "Guarantor assumes full responsibility for uncovered balances...",
      "translation": "You have to pay anything your insurance refuses to cover.",
      "severity": "WARNING"
    }
  ]
}
```